The Journal of Open Source Software

hyper-surrogate: Python-defined hyperelastic materials for finite element solvers

João P. S. Ferreira [1¶]

1 Mechanical Engineering Department, University of Porto, Porto, Portugal ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 21 May 2026
Published: unpublished

Summary

`hyper-surrogate` is a Python package for defining hyperelastic materials and emitting Fortran 90 user subroutines for finite-element solvers. A material can be expressed as a built-in strain-energy function (SEF), a custom symbolic expression on the standard invariants, or a trained data-driven surrogate. All three feed the same chain-rule machinery and produce a self-contained `.f90` file with Cauchy stress and the consistent tangent modulus.

Statement of need

Hyperelastic materials drive simulations across biomechanics, soft robotics, polymers, and biological tissue mechanics, yet bringing a new constitutive model into a finite-element (FE) solver still requires hand-writing a user-material subroutine in Fortran. The author must encode kinematics, the second Piola–Kirchhoff stress, the consistent tangent modulus, a reference-configuration correction, and the solver-specific Voigt layout and objective rate. The task is repetitive, error prone, and disconnected from the only piece of physics the modeller actually authored: the strain-energy function (SEF).

`hyper-surrogate` is built around the idea of having a constitutive model as a single Python object; which can be a built-in class, a one-screen SymPy expression, or a trained surrogate. The kinematics, derivative chain rules, Voigt layout, and reference-configuration handling are framework responsibilities. User responsibilities are limited to defining the material strain-energy function. From that one Python definition, the framework emits a user material `.f90` file exposing Cauchy stress and consistent tangent modulus in the layout the host solver expects. Researchers iterate on the material in Python and the deployment to FE solvers becomes a single function call.

Functionality

- **Symbolic mechanics.** A `SymbolicHandler` exposes the standard invariants; the `Material` abstract base class accepts any SymPy (Meurer et al., 2017) SEF and produces stress and tangent via symbolic differentiation.
- **Fortran emitters.** `UMATHandler` emits an analytical UMAT directly from a symbolic SEF. `HybridUMATEmitter` emits a UMAT whose energy is a trained surrogate and whose stress and tangent are computed analytically in Fortran from the network's gradient and Hessian.
- **Surrogate training.** When the SEF is not available in closed form, a deformation-gradient sampler feeds an MLP, ICNN (Amos et al., 2017) or polyconvex ICNN (Klein et al., 2022) trained on energy and stress with autograd-derived gradient supervision via PyTorch (Paszke et al., 2019).

The Journal of Open Source Software

Minimal pipeline — one material, three UMATs

The same hyperelastic material, expressed three different ways, gives three numerically distinct Fortran UMATs sharing one contract.

Path A — built-in symbolic SEF

```python
from hyper_surrogate import NeoHooke, UMATHandler

# Instantiate one of the catalogue materials with its parameters, then
# emit a complete analytical UMAT in one line.
UMATHandler(NeoHooke({"C10": 0.5, "KBULK": 1000.0})).generate("neohooke.f90")
```

Path B — custom user-defined SEF

```python
import sympy as sp
from hyper_surrogate import Material, UMATHandler


class MyOgdenLike(Material):
    # Default parameter values; users can override any subset at construction.
    DEFAULT_PARAMS = {"mu": 1.0, "alpha": 2.0, "KBULK": 1000.0}

    def __init__(self, parameters=None):
        # Merge user overrides on top of defaults.
        super().__init__({**self.DEFAULT_PARAMS, **(parameters or {})})

    @property
    def sef(self):
        # Closed-form strain-energy function in the standard invariants.
        h = self._handler
        mu, alpha, K = (self._symbols[k] for k in ("mu", "alpha", "KBULK"))
        return ((mu / alpha) * (h.isochoric_invariant1 ** (alpha / 2) - 3)
                + 0.5 * K * (sp.sqrt(h.invariant3) - 1) ** 2)


# The framework derives stress and tangent symbolically; emission is one call.
UMATHandler(MyOgdenLike()).generate("mysef.f90")
```

Path C — data-driven polyconvex surrogate

```python
from hyper_surrogate import (
    NeoHooke, PolyconvexICNN, Trainer, EnergyStressLoss,
    create_datasets, extract_weights, HybridUMATEmitter,
)

# 1. Ground-truth material (could equally be experimental or homogenised data).
material = NeoHooke({"C10": 0.5, "KBULK": 1000.0})

# 2. Sample invariants and energy from physically valid deformations.
train_ds, val_ds, in_norm, energy_norm = create_datasets(
    material, n_samples=4000, input_type="invariants",
    target_type="energy", deformation_mode="combined_compressible",
)
```

```python
# 3. Polyconvex ICNN with one branch per invariant guarantees per-branch
#    convexity in the input invariants.
model = PolyconvexICNN(
    groups=[[0], [1], [2]], hidden_dims=[64, 64, 64], activation="softplus",
)

# 4. Train on energy with autograd-derived stress supervision.
result = Trainer(model, train_ds, val_ds,
                 loss_fn=EnergyStressLoss(alpha=1.0, beta=1.0),
                 max_epochs=2000, patience=200).fit()

# 5. Emit a hybrid UMAT: NN energy + analytical stress and tangent in Fortran.
exported = extract_weights(result.model, in_norm, energy_norm)
HybridUMATEmitter(exported).write("neohooke_polyconvex.f90")
```

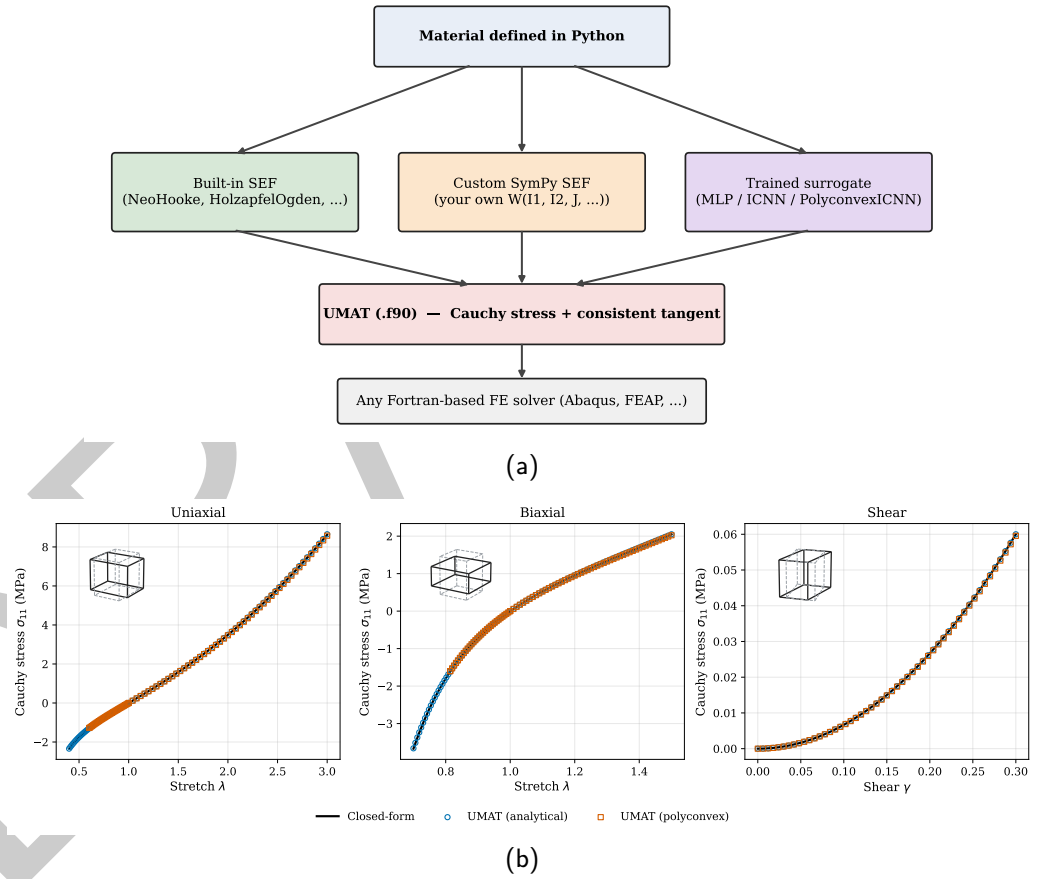

Figure 1: (a) `hyper-surrogate` pipeline — a hyperelastic material defined in Python flows through one of three paths to a Fortran UMAT that any FE solver can consume. (b) Single-element Abaqus round-trip on Neo-Hooke ($C_{10} = 0.5$, $K = 1000$, C3D8H). The analytical UMAT (blue circles) and the polyconvex hybrid UMAT (orange squares) both track the closed-form reference to under 1% in every loading mode; insets show the deformed cube at the representative state of each mode.

Current limitations

`hyper-surrogate` targets isotropic SEFs and fibre-reinforced anisotropic SEFs with up to two fibre families; higher-order anisotropy and inelastic effects (viscoelasticity, damage, plasticity)

are still out of scope. The hybrid neural path requires a scalar (energy-output) network, so stress-output surrogates are routed through the standalone emitter without an analytical consistent tangent. The Fortran emitter follows the Abaqus UMAT convention (*Abaqus User's Manual — UMAT Subroutine*, 2023); deployment to FEAP (Taylor, 2014) and to other Fortran-based codes is feasible through the same Voigt and tangent layout but should be exercised on a per-solver basis.

Research impact

`hyper-surrogate` lowers the cost of moving a new hyperelastic constitutive model from a Python prototype to a production finite-element solver: by collapsing the symbolic-to-Fortran path into a single API call and verifying the emitted user subroutine at the integration-point level against closed-form references (Figure 1), it removes the hand-written-Fortran step that typically gates publication of custom constitutive models for use in commercial FE software.

Acknowledgements

No external funding was received for the development of `hyper-surrogate`.

AI usage disclosure

Generative AI was used for reformulation of sentences in this manuscript. No generative AI tools were used in the development of the core functionalities and architecture of `hyper-surrogate`. Except for unit tests, the further development of `hyper-surrogate` (e.g., optimizing code, docstrings, reviewing, writing documentation, etc.) may involve generative AI tools. All code and documentation are checked and verified by human maintainers before merging into the code base.

References

Abaqus user's manual — UMAT subroutine. (2023). Dassault Systèmes Simulia Corp.

Amos, B., Xu, L., & Kolter, J. Z. (2017). Input convex neural networks. *Proceedings of the 34th International Conference on Machine Learning*, 146–155.

Klein, D. K., Fernandez, M., Martin, R. J., Neff, P., & Weeger, O. (2022). Polyconvex anisotropic hyperelasticity with neural networks. *Journal of the Mechanics and Physics of Solids*, *159*, 104703.

Meurer, A., Smith, C. P., Paprocki, M., Čertík, O., Kirpichev, S. B., Rocklin, M., Kumar, A., Ivanov, S., Moore, J. K., Singh, S., & others. (2017). SymPy: Symbolic computing in Python. *PeerJ Computer Science*, *3*, e103.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 8024–8035.

Taylor, R. L. (2014). *FEAP — a finite element analysis program: User manual*. University of California, Berkeley.